Geometric Control of a Cable-Suspended Payload on a Realistic Quadrotor Model

Reproduction and stress test of Sreenath, Lee & Kumar (CDC 2013)
in MuJoCo with the Skydio X2

Haidar Saad
Master's research, advisor Prof. Andry

Phase 2 report
November 2026

How to read this document. It is written for someone who wants to study the work, not only see the results: every modelling choice is explained with its reason, every result is traced to the experiment that produced it, and Appendix A lists the commands that regenerate each number. Code and result folders are in the public repository `https://github.com/haidar996/suspended-payload-trajectory-tracking`.

Contents

Summary **3**

1 Introduction **3**
1.1 Problem . . . 3
1.2 Why reproduce Paper 1 first . . . 3
1.3 Working method . . . 4

2 Background: model, flatness and controller of Paper 1 **4**
2.1 System model . . . 4
2.2 Differential flatness . . . 4
2.3 Geometric controller . . . 5

3 Simulation framework **5**
3.1 Design decisions . . . 5
3.2 The Skydio X2 model . . . 5
3.3 Cable models used in Phase 2 . . . 5
3.4 Actuation stack . . . 7
3.5 Realism levels . . . 7
3.6 Benchmark protocol . . . 7
3.7 Software organisation and validation philosophy . . . 8

4 Phase 1: validation of the simulation core **8**

5 Phase 2: implementing Paper 1 **9**
5.1 Flatness map (step 2.1, gates F1–F6) . . . 9
5.2 Choosing the gains (finding F3) . . . 9
5.3 Controller gates (step 2.2, K1–K7) . . . 10

6 Experiments with Paper 1 **10**
6.1 E1.1 The flatness study on the ellipse (step 2.3) . . . 10
6.2 E1.2 Recovery from a near-flipped start (step 2.4) . . . 12
6.3 E1.3 Benchmark subset T0–T3 (step 2.5) . . . 12
6.4 E1.4a A cable that can go slack (step 2.6a) . . . 13
6.5 E1.4b Cable length (step 2.6b) . . . 14
6.6 E1.4c Realistic actuation and sensing (step 2.6c) . . . 15

7 Summary of findings **16**

8 Discussion **17**
8.1 What the results say about Paper 1 . . . 17
8.2 Limits of this study . . . 17

8.3 Gaps and next steps 17

A Reproducing the results **18**

B Repository map **19**

C Glossary **19**

Summary

A quadrotor carrying a load on a cable is an underactuated system: the four rotors act on the drone, and the load moves only through the cable. Sreenath, Lee and Kumar (CDC 2013), called *Paper 1* here, showed that this system is differentially flat with the load position and the drone yaw as flat outputs, and designed a geometric controller that tracks the load position with almost-global stability guarantees. Their proofs and simulations assume a rigid massless cable that is always taut, unlimited rotor thrust, perfect state knowledge and instantaneous actuators.

We reimplemented the flatness map and the controller from the equations of the paper and ran them on a realistic simulated drone, the Skydio X2 from MuJoCo Menagerie, whose four rotors each produce between 0 and 13 N. Under the paper's own assumptions every result reproduces: the flatness map agrees with an independent implementation to 10^{-13}, references are tracked to below 0.2 mm, and the drone recovers from a 178° flip. We then removed the assumptions one at a time, and each removal broke something specific:

- Rotor limits cap the attitude loop at about 30 rad/s, give the X2 a maximum speed on the paper's own ellipse (period $T_0^* = 2.12$ s), and make recovery from flips of 20° or more chaotic.
- A cable that can go slack makes the 178° recovery diverge, even with unlimited thrust: the paper's recovery depends on the cable pushing the load.
- With the published gain scaling, a 1 m cable crashes the X2 on a 15° initial swing.
- Motion-capture-level velocity noise crashes every run, because the controller differentiates noisy states without an estimator; a 35 ms motor lag breaks swing recovery, and gains slow enough for the lag can no longer damp the swing.

These results define the gaps that the next phases address: control that respects rotor limits, recovery that handles a slack cable, state estimation, and swing damping that is compatible with motor lag.

1 Introduction

1.1 Problem

Transporting a suspended load with a multirotor is attractive because the cable keeps the load away from the rotors and lets a small drone carry objects of many shapes. The price is a coupled, underactuated dynamics: the drone has six degrees of freedom and four inputs, and the load adds two more degrees of freedom (the direction of the cable) with no input of their own. Moving the drone makes the load swing; letting the load swing makes the drone's position hard to hold. The research question of this project is how to make the *load* follow a desired trajectory accurately and robustly, eventually with learning-based methods, under realistic conditions.

1.2 Why reproduce Paper 1 first

Paper 1 is the reference model-based solution. Its two ideas, differential flatness and geometric control on $SO(3) \times S^2$, are used by most later work, including the reinforcement-learning papers planned for the later phases. Reproducing it gives us (i) a verified simulation framework, (ii) a strong baseline, and (iii) a map of where the classical theory stops working, which is where a contribution can be made.

1.3 Working method

The project follows a fixed research workflow: before any implementation, the literature is searched for each modelling choice; the choice is discussed and agreed; it is recorded in a design log (`SIMULATION_DESIGN.md`, decisions D1–D12 and findings F1–F10); and implementation then proceeds in small steps, each with explicit pass/fail gates, automated tests, and a commit only after approval. Every experiment writes a self-contained result folder with its configuration, git commit, metrics, raw data and figures. This report summarises Phase 1 (simulation core) and Phase 2 (Paper 1).

2 Background: model, flatness and controller of Paper 1

2.1 System model

We use the notation of Paper 1. The drone has mass m_Q, inertia J about its centre of mass, position $x_Q \in \mathbb{R}^3$, attitude $R \in SO(3)$ and body angular velocity Ω. The load is a point mass m_L at x_L, attached by a massless cable of length l at the drone's centre of mass. The unit vector $q \in S^2$ points from the drone to the load, so $x_Q = x_L - l\,q$, and $\omega = q \times \dot{q}$ is the cable's angular velocity. The rotors produce a collective thrust f along the body axis $R\mathbf{e}_3$ and a body moment M. While the cable is taut with tension T,

$$m_L(\ddot{x}_L + g\mathbf{e}_3) = -Tq, \tag{1}$$
$$m_Q(\ddot{x}_Q + g\mathbf{e}_3) = fR\mathbf{e}_3 + Tq, \tag{2}$$
$$J\dot{\Omega} + \Omega \times J\Omega = M, \qquad \dot{R} = R\,\hat{\Omega}, \tag{3}$$

where $\hat{\cdot}$ maps a vector to its skew-symmetric matrix. The configuration space is $SE(3) \times S^2$ and the system has eight degrees of freedom with four inputs.

2.2 Differential flatness

A system is *differentially flat* if a set of outputs (the flat outputs) exists such that all states and inputs are algebraic functions of these outputs and finitely many of their derivatives. For this system the flat outputs are the load position x_L and the drone yaw ψ. The map, implemented in `sim/flatness.py`, is:

$$u = \ddot{x}_L + g\mathbf{e}_3, \qquad q = -u/\|u\|, \qquad T = m_L\|u\|, \tag{4}$$
$$F = fR\mathbf{e}_3 = (m_Q + m_L)\,u - m_Q l\,\ddot{q}, \tag{5}$$
$$b_3 = F/\|F\|, \quad R = [\,b_1\ b_2\ b_3\,] \text{ from } b_3 \text{ and } \psi, \tag{6}$$
$$\Omega = (R^\top \dot{R})^\vee, \qquad M = J\dot{\Omega} + \Omega \times J\Omega. \tag{7}$$

The attitude needs the fourth derivative of x_L (through $\ddot{q}$), the body rate the fifth and the moment the sixth. The map is singular when $\|u\| \to 0$ (the load in free fall, zero tension) and when the thrust direction is parallel to the yaw heading.

Exact derivatives by jet arithmetic. Instead of symbolic algebra (too slow, 7 ms per evaluation) or finite differences (inaccurate at sixth order), we propagate truncated Taylor series ("jets") through the map. A jet stores a value and its first n derivatives; products use the Leibniz rule and normalisation uses the series of $1/\sqrt{\cdot}$. All six derivatives are then exact to round-off at 0.1 ms per evaluation (`sim/jet.py`).

2.3 Geometric controller

Paper 1 builds three flight modes, each on top of the previous one. Our implementation is `sim/controllers/geometric_paper1.py`.

(a) Drone attitude (Eq. 21 of Paper 1). With attitude error $e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee$ and rate error $e_\Omega = \Omega - R^\top R_d \Omega_d$,

$$M = -K_R e_R - K_\Omega e_\Omega + \Omega \times J\Omega - J(\hat{\Omega} R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d). \tag{8}$$

(b) Cable attitude (Eqs. 25–32). With cable errors $e_q = \hat{q}^2 q_d$ and $e_{\dot{q}} = \dot{q} - (q_d \times \dot{q}_d) \times q$ (a tangent vector at q), the desired thrust vector is

$$F = F_n - F_{pd} - F_{ff}, \qquad F_{pd} = -k_q e_q - k_\omega e_{\dot{q}}, \tag{9}$$

with feed-forward $F_{ff} = m_Q l \langle q, q_d \times \dot{q}_d \rangle (q \times \dot{q}) + m_Q l (q_d \times \ddot{q}_d) \times q$. The commanded attitude R_c has third axis $F/\|F\|$, and $f = F \cdot R\mathbf{e}_3$.

(c) Load position (Eqs. 36–38). With $e_x = x_L - x_{L,d}$ and $e_v = \dot{x}_L - \dot{x}_{L,d}$,

$$A = -k_x e_x - k_v e_v + (m_Q + m_L)(\ddot{x}_{L,d} + g\mathbf{e}_3) + m_Q l \langle \dot{q}, \dot{q} \rangle q, \qquad q_c = -A/\|A\|, \qquad F_n = \langle A, q \rangle q. \tag{10}$$

Time-scale separation. The stability argument treats the three loops as nested: the attitude loop is fast enough to be at its target, the cable loop is fast relative to the position loop. In Paper 1 this is a singular-perturbation argument with a small parameter ε scaling the attitude gains. In practice it means the loop bandwidths must be well separated, and Section 5.2 shows how much separation is needed.

Rates of the computed commands. Modes (b) and (c) need $\dot{q}_c, \ddot{q}_c$ and $\Omega_c, \dot{\Omega}_c$. We take them from the flatness map of the reference (exact feed-forward). Two ablations, filtered numerical differentiation and zero rates, are kept to show why this matters (Section 5.3).

3 Simulation framework

3.1 Design decisions

Twelve design decisions were made from the literature before any code was written. Table 1 gives the ones used in this report; the design log contains the sources and the alternatives that were rejected.

3.2 The Skydio X2 model

The X2 is loaded from the Menagerie files and modified only in code (through MuJoCo's `MjSpec` editing API), so the vendored files stay identical to upstream. Its properties are read back from the compiled model, never hard-coded.

Three settings are applied in code: the physics step and integrator (D5), ground contacts disabled (the drone starts at 2 m and no experiment needs the ground), and the air-drag model switched off at realism level L0 and on at L1 (D10).

3.3 Cable models used in Phase 2

C1, rigid massless link. The payload body is a child of the drone, connected by a ball joint at the attachment point, with its mass placed at distance l. A ball joint has no singularity at any swing angle. This is exactly Paper 1's model: the link can pull and push.

ID	Decision	Main reason
D1	Simulator: MuJoCo 3.17 (CPU now, MJX later for RL)	accurate contact-free rigid-body dynamics; same model on GPU later
D2	Drone: Skydio X2 from MuJoCo Menagerie (vendored, unmodified)	no real drone available; maintained, realistic inertia and rotor layout
D3	Cable models C1–C6 as an experimental variable	literature uses rigid, elastic, chained and rod models; C1 is Paper 1, C3 is Paper 3
D5	Physics 1 ms RK4; inner loop 500 Hz; outer loop 100 Hz	no contacts, so RK4 is the most accurate choice
D6	Payload P1 point mass 0.2 kg (P3 rigid cylinder later)	matches Paper 1; P3 needed for Paper 3
D7	Cable length 0.5 m nominal; 0.25 and 1.0 m in sweeps	covers the range used in the literature
D8	Actuation stack SRT / CTBM / CTBR; motor lag 35 ms in the main benchmark	CTBR is the interface used by RL papers and real autopilots
D9	Benchmark trajectories T0–T5, metrics, success rule	no shared benchmark exists for payload control
D10	Realism levels L0 (ideal), L1 (noise, delay, drag), L2 (no payload sensor)	tests reality gap step by step
D11	Randomisation ranges	for robustness tests and RL training
D12	Tuning protocol	fair comparison between controllers (Phase 3)

Table 1: Design decisions referenced in this report.

Property	Value
Mass m_Q	1.325 kg
Centre of mass (body frame)	$(0, 0, 0.054)$ m
Inertia about CoM	$J = \begin{bmatrix} 0.0367 & 0 & -0.0021 \\ 0 & 0.0254 & 0 \\ -0.0021 & 0 & 0.0605 \end{bmatrix}$ kg m^2
Rotor positions relative to CoM	$(\pm 0.14, \pm 0.18)$ m
Thrust per rotor	0 to 13 N (total 52 N, thrust-to-weight 3.5 with the payload)
Yaw torque per newton of thrust	± 0.0201 m
Hover thrust per rotor (with 0.2 kg payload)	3.74 N

Table 2: Skydio X2 properties as compiled in our simulation.

C2, inextensible and slack-capable. The payload is a free body. A MuJoCo spatial tendon between the attachment site and the payload is limited to $0 \leq \text{length} \leq l$. When the payload is closer than l, no force acts on it (the cable is slack); at l the limit becomes an active constraint that pulls, never pushes. The softness of the limit constraint (time constant 4 ms) was chosen and validated in step 2.6a (Section 6.4).

3.4 Actuation stack

The controller output passes through the same chain a real drone uses:

$$\text{controller} \rightarrow \text{CTBR (thrust + body rates)} \rightarrow \text{rate loop} \rightarrow \text{CTBM (thrust + moments)} \rightarrow \text{mixer and saturation} \rightarrow \text{motor lag} \rightarrow \text{X2}$$

Paper 1 outputs thrust and moments directly (CTBM). The **mixer** inverts the 4×4 allocation matrix $[f, \tau_x, \tau_y, \tau_z]^\top = B\,[f_1, \ldots, f_4]^\top$. If the result violates 0–13 N it desaturates with PX4 priorities: roll and pitch torque are kept first (scaled only if they alone are infeasible), then collective thrust is moved to the closest feasible value, and yaw torque is reduced last. The **motor lag** is a first-order filter on each rotor's thrust, implemented as MuJoCo actuator dynamics so it is integrated by the physics engine itself (finding F2: the `filter` dynamics type is accurate under RK4, `filterexact` is not).

3.5 Realism levels

At L0 the controller sees the exact state. At L1 (D10) it sees motion-capture-like measurements: independent Gaussian noise of 1 cm on positions, 5 cm/s on velocities, 1° on attitude and 0.05 rad/s on the gyro, applied at every 500 Hz tick; commands are delayed by 10 ms; MuJoCo's fluid model adds air drag on drone and payload. The cable direction the controller uses is rebuilt from the noisy drone and payload positions, as an estimator based on motion capture would do.

3.6 Benchmark protocol

The benchmark (D9) uses the load position as the tracked quantity. Its trajectories are:

ID	Trajectory	Levels
T0	Hover with an initial swing, or a lateral gust on the payload for 0.5 s	swing 15°, 30°; gust $0.25g \cdot m_L$ (0.49 N) and severe 2 N
T1	Rest-to-rest 1 m move, snap-smooth polynomial	4, 2, 1 s
T2	Circle $r = 1$ m after a smooth speed ramp	peak acceleration $0.25g$, $0.5g$, $1g$
T3	Figure-8 (Lissajous 1:2), 1×0.5 m	period 15, 10, 5 s

Table 3: Benchmark trajectories used in Phase 2 (T4 random set and T5 slack tests are defined in D9; T5(a) is used in Section 6.4).

The metrics are the payload RMSE, maximum and final error; maximum, RMS and residual swing; *dynamic swing*, the angle between the actual cable direction and the direction the flatness map predicts for the reference (it separates the tilt a trajectory needs from unwanted oscillation); settling time into a 5 cm band held for 1 s; minimum tension and slack fraction; rotor saturation fraction; maximum tilt; and controller time per step. A run is a *success* if the drone does not crash (altitude below 0.3 m, tilt above 90° or numerical failure), the payload error stays below 0.5 m, the swing stays below 60°, and the payload never rises above the drone.

3.7 Software organisation and validation philosophy

The code is a small Python package (`sim/`) with one module per concern: configuration, model building, actuators, state extraction, trajectories, metrics, logging, flatness, controllers, realism and the closed-loop runner. Each implementation step has a `checks_*.py` module with numbered gates, a test file that runs the gates, and an experiment script that writes a result folder. Gates are physical or numerical facts with a stated tolerance, for example "the swing period matches pendulum theory within 0.5%". Where a fixed tolerance would be arbitrary, convergence is tested instead: the error must shrink by the expected factor when the timestep is halved. At the end of Phase 2 the suite contains 66 tests, all passing.

4 Phase 1: validation of the simulation core

Before any controller was written, each layer of the simulator was checked against physics it must obey.

Drone model (gates G1–G5). At exact hover thrust the drone does not drift; in free fall it follows $-\frac{1}{2}gt^2$ to 4×10^{-14} m; halving the timestep changes a 1 s manoeuvre by 3×10^{-10} m. The air-drag check led to finding F1: MuJoCo's fluid drag is quadratic in speed, about 0.10 N at 2 m/s for the X2, while real rotor drag is roughly linear and larger at low speed. Drag is therefore covered by randomisation in later tests rather than trusted as a precise model.

Actuators (A1–A6). The mixer reproduces any feasible wrench to 3×10^{-14}; infeasible commands are desaturated in the stated priority order; the motor lag matches the analytic first-order response to 6×10^{-9} N.

Cable C1 with payload P1 (P1–P7). The static tension equals $m_L g$ exactly. The small-swing period with a free drone matches the two-body pendulum formula $2\pi\sqrt{m_Q l/(g(m_Q + m_L))}$ within 0.012% for 0.25, 0.5 and 1.0 m (Figure 1). Energy drifts by 1.3×10^{-7} J in a 2 s free swing, the cable length stays constant to 3×10^{-15} m, and the state-extraction errors shrink by a factor of four per timestep halving, as RK4 requires.

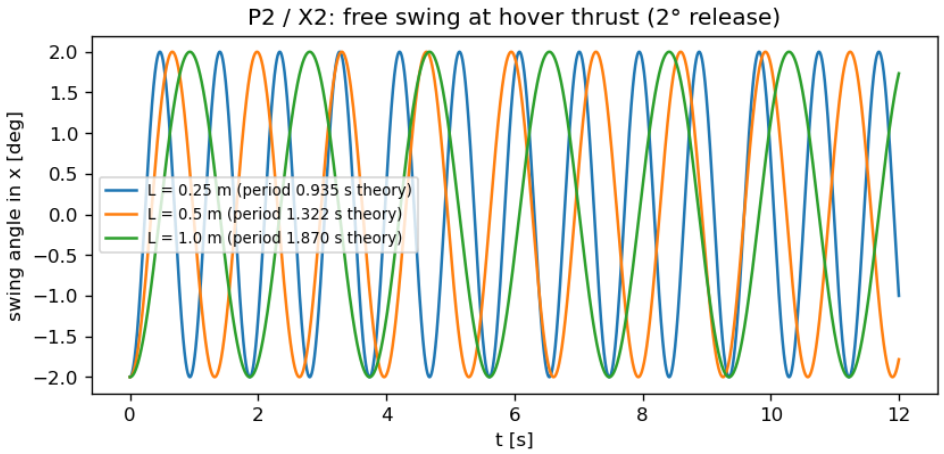


Figure 1: Gate P2: swing of the C1 payload under a hovering drone for three cable lengths, compared with the two-body pendulum period.

Trajectories and metrics (M1–M5). Reference trajectories are built from polynomials and complex exponentials so that derivatives up to sixth order are analytic; they agree with a symbolic (sympy) oracle to 1.4×10^{-11} relative error. Metrics are verified on signals with known answers, and the logger is bit-reproducible.

5 Phase 2: implementing Paper 1

5.1 Flatness map (step 2.1, gates F1–F6)

The flatness map is exact at hover, satisfies the equations of motion with a residual of 10^{-14}, and its derivatives converge at the expected rate. As an independent check it agrees with Udaan, an open-source implementation of the same map, to 1.1×10^{-13}. The strongest test is an *open-loop replay*: the drone is placed exactly on the flat state of a reference and the flatness wrench is applied with no feedback. The real MuJoCo dynamics then follow the reference within 1 cm for 3 to 6 s before the unstable error growth takes over (Figure 2). This confirms that the map, the X2 model and the actuator chain are mutually consistent.

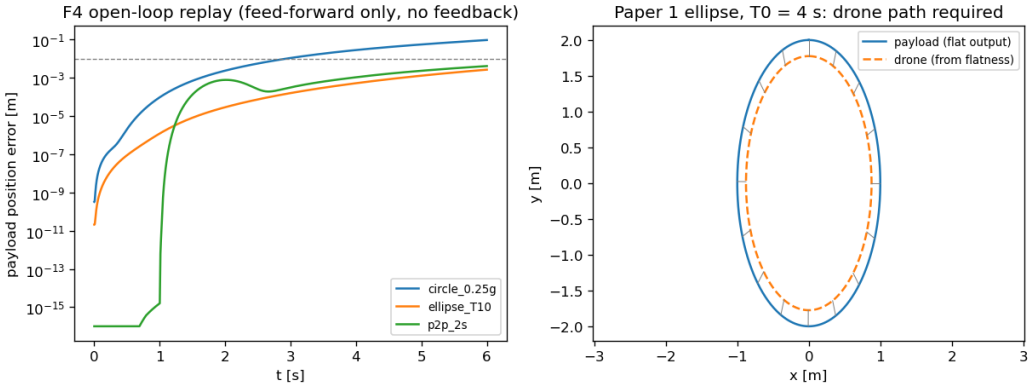


Figure 2: Step 2.1: open-loop replay of flatness wrenches in MuJoCo for several references, and the derivative convergence checks.

5.2 Choosing the gains (finding F3)

Paper 1 does not publish usable gains for this drone. We parametrise the three loops by their bandwidths $\omega_x, \omega_q, \omega_R$ and damping ratio $\zeta = 0.9$:

$$k_x = (m_Q{+}m_L)\,\omega_x^2,\ k_v = 2\zeta(m_Q{+}m_L)\,\omega_x, \quad k_q = m_Q l\,\omega_q^2,\ k_\omega = 2\zeta m_Q l\,\omega_q, \quad K_R = \omega_R^2 J,\ K_\Omega = 2\zeta\omega_R J.$$

Scaling the attitude gains by the inertia gives every axis the same bandwidth. Two constraints then fix the values.

Separation. With only 3× between loops (2 / 6 / 20 rad/s) the coupled system is lightly damped: a 1 m step settles in 3.65 s instead of the 2.13 s predicted by the linear design. With 2× it oscillates indefinitely, and a cable loop only 3× below the attitude loop (10 / 30) is unstable. With at least 5× the closed loop matches the linear prediction.

Rotor limits. At 40–60 rad/s attitude bandwidth a 30° swing saturates the X2 rotors on almost every tick and the drone diverges, while the same gains converge with unlimited thrust. About 30 rad/s is therefore the ceiling.

F3: gains used throughout Phase 2

$\omega_x, \omega_q, \omega_R = 1.5,\ 7.5,\ 30$ rad/s, i.e. $k_x = 3.43$, $k_v = 4.12$, $k_q = 37.3$, $k_\omega = 8.94$, $\mathrm{diag}(K_R) = (33.0, 22.9, 54.5)$, $\mathrm{diag}(K_\Omega) = (1.98, 1.37, 3.27)$ in SI units. The time-scale separation that the theory needs and the thrust the X2 has leave very little room: these are the fastest gains that satisfy both.

5.3 Controller gates (step 2.2, K1–K7)

Gates that test the theory itself (K1, K2, K5) use an *ideal actuator* option that removes rotor limits, because Paper 1 assumes unlimited inputs. The others use the real X2 limits.

Gate	Test	Result
K1	Attitude mode, 60° initial error	error decays at 28.4/s vs 27/s predicted
K2	Cable-attitude mode, 120° initial cable error	decays at 7.6/s vs 6.75/s predicted
K3	Position mode, 1 m step	settles in 2.87 s vs 2.83 s from linear design
K4	Tracking T1 2 s, T2 0.5g, T3 10 s	max error ≤ 0.17 mm
K5	Faster attitude loop (15, 30, 60 rad/s)	error decreases monotonically, as singular perturbation predicts
K6	Cable-rate error form	tangent to S^2; the wrong (angular-velocity) form leaves a 179° residual swing
K7	Source of command derivatives	flatness 0.05 mm, numerical differentiation 12.5 mm, zero 0.67 m RMSE

Table 4: Step 2.2 controller gates.

K7 is worth stressing: the feed-forward from the flatness map is not a refinement but a requirement. Without it the same controller misses a circle by more than half a metre.

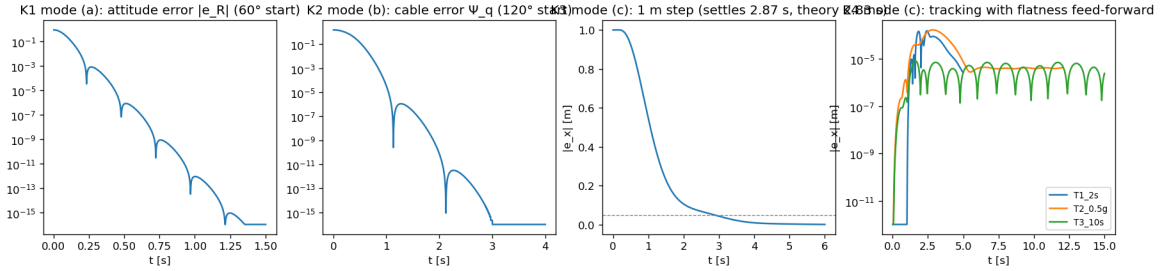


Figure 3: Step 2.2: attitude-mode decay (K1), cable-mode decay (K2), 1 m step (K3) and tracking errors (K4).

6 Experiments with Paper 1

6.1 E1.1 The flatness study on the ellipse (step 2.3)

Setup. Paper 1's Fig. 3 uses the load trajectory $x_L(t) = (A_x \sin \frac{2\pi t}{T_0},\ A_y \cos \frac{2\pi t}{T_0},\ A_z)$ with $\psi = 0$ and periods T_0 from 10 s down to 1 s. We use $A_x \times A_y = 0.5 \times 1.0$ m as the main case, 1.0×2.0 m as a second case, and A_z at the payload's hover height. The study has three parts: (A) reproduce Fig. 3 from the flatness map; (B) find, by bisection on T_0, the shortest period for which every rotor thrust required by the flatness map stays in 0–13 N; (C) run the closed loop for three full periods on the X2 and with the ideal actuator, starting exactly on the flat state.

Part A. The load path is identical for every T_0; the drone paths are not (Figure 4). As T_0 decreases the drone flies a smaller, lower, peanut-shaped path *inside* the load's ellipse, because the load swings outward and the drone has to lean against it. Peak tilt grows from 20° at 3 s to 54° at 1.5 s and exceeds 90° below about 1.25 s, where the drone would have to fly partly upside down. Our first gate assumed the drone's path grows with speed; the simulation showed the opposite, and the gate was replaced by "required tilt and torque grow as T_0 decreases".

Part B. Table 5 gives the feasibility boundary. Below it the required thrust explodes: at

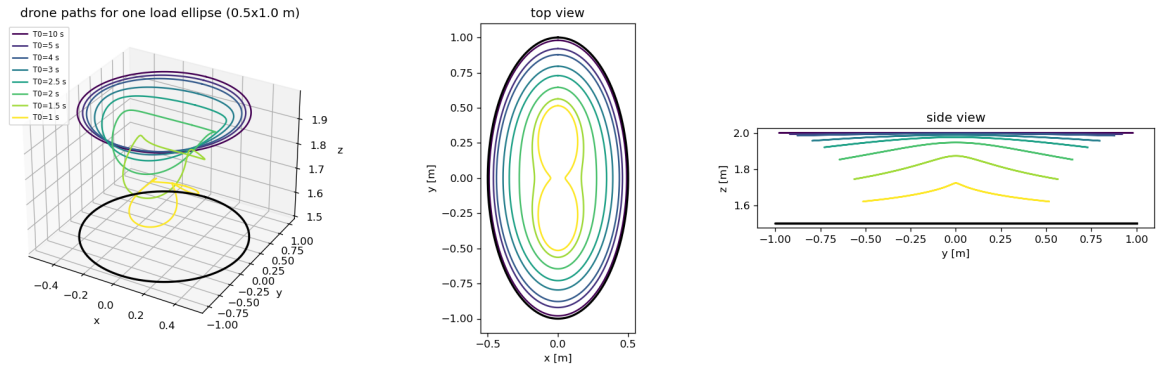


Figure 4: Paper 1 Fig. 3 reproduced: drone paths (colour) required for one load ellipse (black), $T_0 = 10 \ldots 1$ s.

$T_0 = 1.5$ s the flatness map asks for thousands of newtons per rotor. Across a size sweep (A_y from 0.25 to 3 m at aspect 1:2) T_0^* grows monotonically from 1.35 s to 2.92 s.

Ellipse	T_0^*	Load peak speed	Load peak acceleration
0.5×1.0 m	2.12 s	3.0 m/s	8.8 m/s^2
1.0×2.0 m	2.61 s	4.8 m/s	11.6 m/s^2

Table 5: X2 feasibility boundary on Paper 1's ellipse (rigid cable 0.5 m, payload 0.2 kg).

Part C. Above T_0^* the X2 behaves exactly like the ideal actuator, with no saturation, and errors range from 1 μm at 10 s to about 1.6 mm close to the boundary. Slightly below it (2.0 s for the main ellipse, 2.5 s for the second) the rotors saturate on 42–51% of ticks, yet the prioritised mixer keeps the error below 1.7 mm. Further below, the X2 diverges. At $T_0 = 1.5$ s *even the ideal actuator fails* (19 cm and 2.2 m errors): the reference needs about 62 rad/s body rates, which a 30 rad/s attitude loop cannot follow (Figure 5).

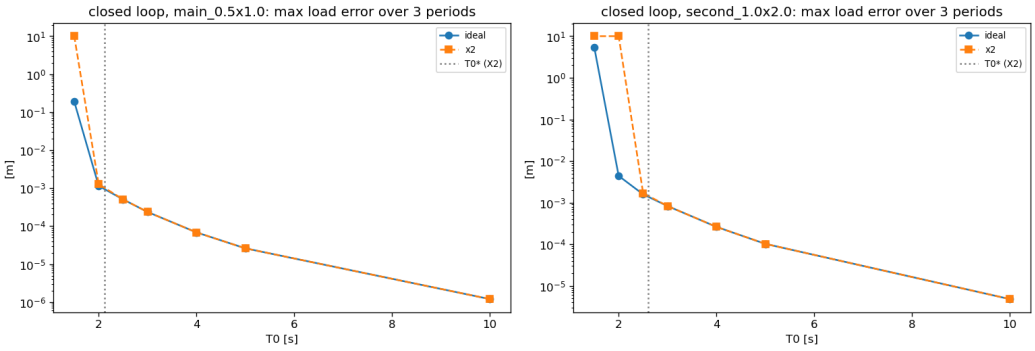


Figure 5: Closed-loop maximum load error over three periods vs T_0, X2 and ideal actuator; dotted line: T_0^*.

F4: two speed limits

The X2 can fly Paper 1's ellipse only down to $T_0^* = 2.12$ s (0.5×1.0 m), a load speed of 3 m/s. A second, lower limit comes from the controller: with attitude bandwidth capped at 30 rad/s by F3, references faster than about 1.5 s cannot be tracked even with unlimited thrust.

6.2 E1.2 Recovery from a near-flipped start (step 2.4)

Setup. Paper 1 (Figs. 4–5) starts the drone flipped 178° about the y axis, the cable at 178° and the load far from its reference, without giving the numbers. We rotate drone and cable by the same angle θ about y, start everything at rest with the load 2 m from the reference along $(1, -1, -1)/\sqrt{3}$, and use the main ellipse at $T_0 = 5$ s as the reference, for 10 s. Recovery means entering a 5 cm band and staying there for at least 2 s.

Paper 1 reproduced. With the ideal actuator and $\theta = 178°$ all three errors converge (Figure 6): the load is inside 5 cm at 5.1 s, the final error is 0.07 mm, and the drone loses 0.72 m of height. The tension plot shows something the paper does not discuss: for about the first second the rigid link *pushes* the load (tension down to −2.8 N, 9% of the run).

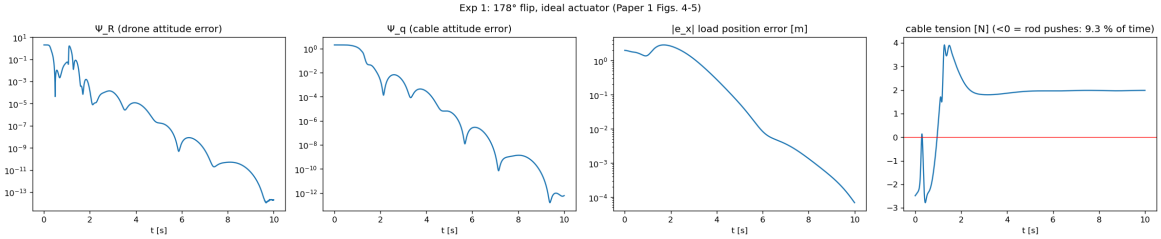


Figure 6: Recovery from a 178° flip with unlimited thrust and the rigid cable: attitude, cable and position errors, and the cable tension.

On the X2. With 0–13 N rotors the result depends on θ in an irregular way (Figure 7). A 2 m position error alone ($\theta = 0$) recovers in 3.1 s with 2% saturation, and 10° also recovers cleanly. From 20° the controller demands 15–46 N m against roughly 2 N m available, the rotors are pinned at their limits, and drone and load tumble. Outcomes then change with 1° differences (30° fails, 31° recovers); recovered runs lose 1–9 m of height and failed runs fall up to 50 m.

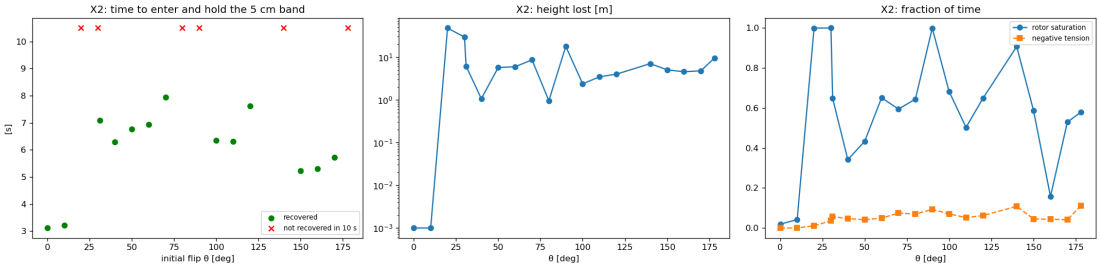


Figure 7: X2 flip-angle sweep: recovery time (red crosses: not recovered in 10 s), height lost, rotor saturation and negative-tension fraction.

F5: recovery under rotor limits

Paper 1's almost-global recovery relies on unlimited torque and on a cable that can push. On the X2, recovery from flips of 20° or more is unreliable (chaotic in the initial angle) and costs metres of height.

6.3 E1.3 Benchmark subset T0–T3 (step 2.5)

All 13 benchmark cases of Table 3 were run on the X2 and with the ideal actuator, after a feasibility pre-check with the flatness map. Every case whose reference is feasible succeeds, and on these cases the X2 and the ideal actuator give identical numbers: tracking errors are at most 0.45 mm on T1–T3, the 15° and 30° initial swings settle in 1.6 and 2.0 s (about 1.1 and 1.4 pendulum periods), and the mass-scaled gust settles in 1.25 s. Only two cases fail: the 1 m move in 1 s,

whose reference needs ±570 N per rotor, and the severe 2 N gust (a $1g$ push on a 0.2 kg payload), which no controller can keep within the 0.5 m success limit.

The dynamic-swing metric behaves as designed: on the tracking cases the swing reaches 45° while the dynamic swing stays below 0.01°. All of that swing is what the trajectory requires, none is unwanted oscillation.

The original D9 gust (2 N) was revised during this step: it was replaced by $0.25g \times m_L$ so that it scales with payload mass, and 2 N was kept as a "severe" level. Controller time is 1.4 ms per step in Python, including the flatness feed-forward, which fits in the 2 ms period at 500 Hz with little margin.

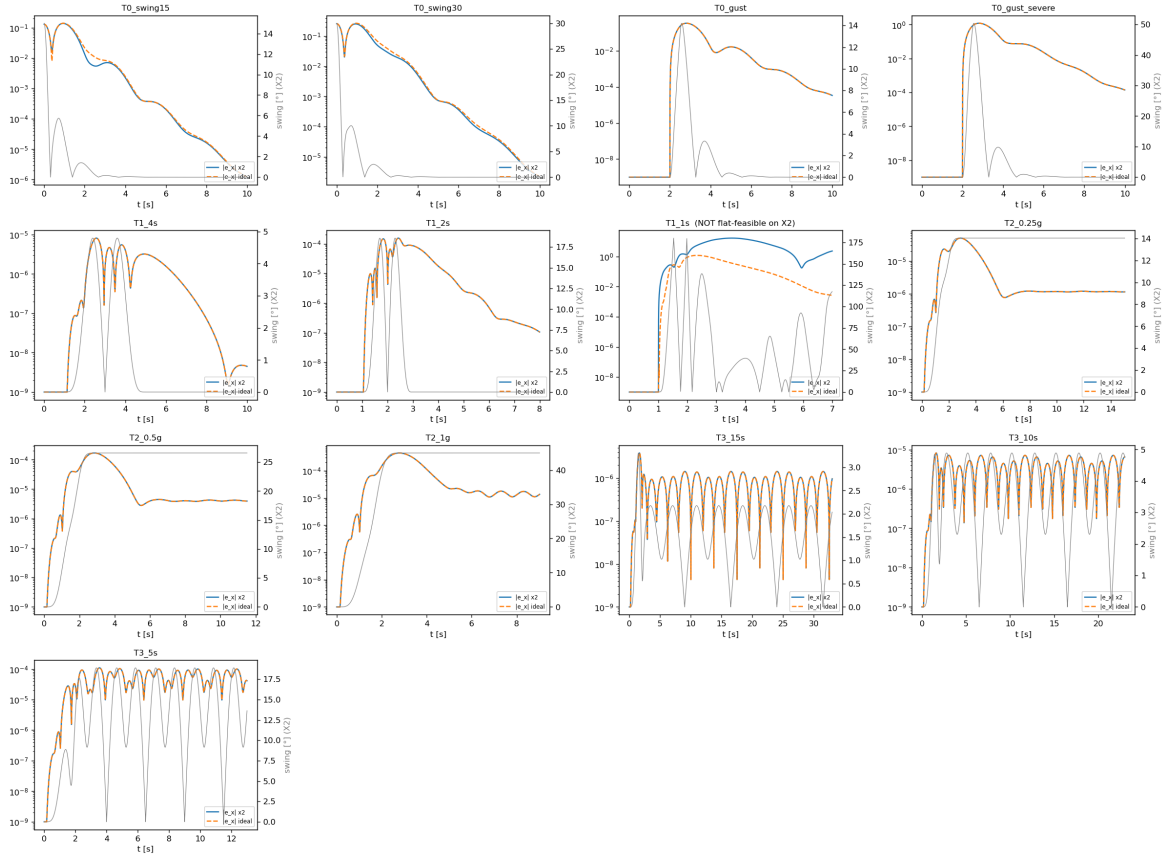


Figure 8: Benchmark subset: load error (log scale, solid X2, dashed ideal actuator) and swing angle (grey) for each case.

6.4 E1.4a A cable that can go slack (step 2.6a)

Validation of C2 (gates S1–S6). While taut, C2 reproduces C1: hover stretch 0.01 mm, swing period within 0.008% of theory, and payload paths within 0.10 mm of C1 over a 30° swing. When the payload is thrown upward at 3 m/s it goes slack twice, the tension is exactly zero while slack, the payload follows a ballistic arc (acceleration error 2×10^{-10}), and the tension is never negative.

F6: what a snap means in an inextensible cable

When a slack inextensible cable becomes taut it stops the payload's relative motion instantly: physically this is an impulse, with no finite peak force. The peak tension MuJoCo reports (186 to 509 N) is set only by the softness of the constraint and the timestep, so it is not a physical result and is never reported for C2. The *impulse* is physical (about 0.64 N s, the payload's momentum change); it converges between 0.5 and 0.25 ms (3%) but is under-resolved by about 20% at 1 ms. C2 runs therefore use a 0.5 ms timestep (an agreed exception to D5). Realistic snap forces need an elastic cable (C3, Paper 3).

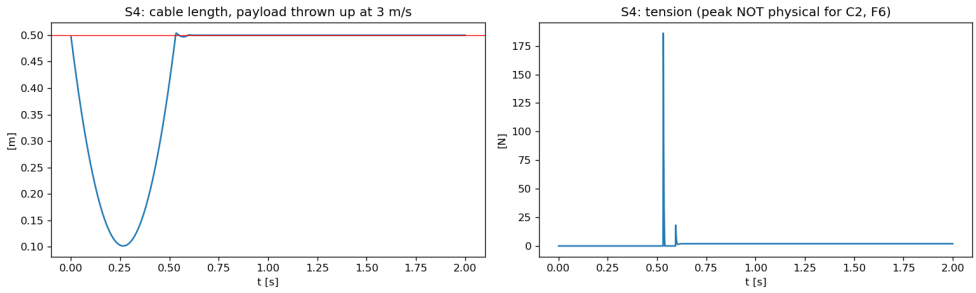


Figure 9: Gate S4: cable length and tension when the payload is thrown upward at 3 m/s from hover.

Paper 1 on C2. On every benchmark case where the cable stays taut, C2 gives the same result as C1 within 0.03 mm. In the T5(a) test (payload thrown up at 3 m/s) the cable is slack for 0.88 s, the payload passes above the drone, and the controller still recovers (maximum error 0.46 m, settled in 2.44 s). The decisive result is the 178° recovery: *with unlimited thrust it now diverges*. With the load above the drone and the cable slack, thrust no longer acts on the load; the controller keeps commanding it, the unlimited actuator launches the drone about 21 m in 0.1 s, the cable snaps taut at over 100 N and the run diverges. On the X2 the flip sweep recovers in 7 of 19 angles instead of 14 of 19. The descent faster than g of T5(b) cannot even be posed: its reference makes the required tension negative, which is exactly where the flatness map is singular.

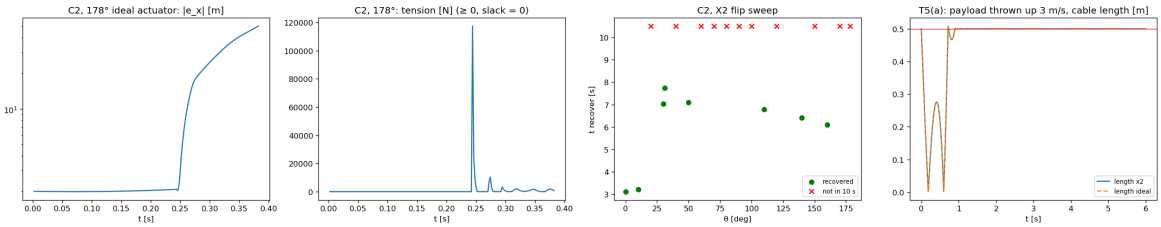


Figure 10: Paper 1 on the slack-capable cable: divergence of the 178° recovery with unlimited thrust, the X2 flip sweep, and the T5(a) throw test.

F7: the always-taut assumption

Paper 1's controller is fine while the cable is taut and survives a short slack phase, but its large-angle recovery relies on the cable being able to push the load.

6.5 E1.4b Cable length (step 2.6b)

The benchmark and the feasibility boundary were repeated with cable lengths of 0.25 m and 1.0 m, keeping the same bandwidths. The concern before the run was the short cable: its free swing frequency (6.7 rad/s) is close to the 7.5 rad/s cable loop, which could break time-scale separation. It did not: 0.25 m succeeds on every feasible case. The problem appeared at 1.0 m, where the 15° initial swing flips the X2 (30% saturation) and the 30° swing fails even with unlimited thrust.

The reason is in the gain scaling. Keeping a fixed bandwidth means $k_q = m_Q l\,\omega_q^2$ grows with l, so the restoring *force* for the same swing angle doubles from 0.5 to 1.0 m: a 15° swing on 1 m asks for about 19 N sideways, which in turn asks for a large tilt and saturates the rotors.

	0.25 m	0.5 m	1.0 m
Free swing frequency	6.7 rad/s	4.7 rad/s	3.4 rad/s
T_0^*, main ellipse	1.94 s	2.12 s	2.26 s
T0 15° swing (X2)	settles 1.8 s	settles 1.6 s	crash
T0 30° swing (X2)	settles 2.2 s	settles 2.0 s	crash (ideal actuator also fails)
Feasible tracking cases	all succeed	all succeed	all succeed

Table 6: Effect of cable length with unchanged bandwidths (finding F8).

Two smaller observations: a longer cable lowers the X2's maximum speed (T_0^* increases), and the flatness feasibility check is conservative, since at 1.0 m two cases flagged as infeasible (T1 2 s and T2 $1g$) were still tracked with brief saturation.

6.6 E1.4c Realistic actuation and sensing (step 2.6c)

A linear pre-check. For one attitude axis with a 35 ms motor lag, the PD loop $J\ddot{\theta} = \tau$ has the characteristic polynomial $\tau_m s^3 + s^2 + 2\zeta\omega_R s + \omega_R^2$. Table 7 shows its least-damped pole, with and without an extra 10 ms delay (first-order Padé). At our 30 rad/s the loop is barely damped with lag and unstable with lag plus delay. A lag-compatible attitude loop has to be near 10 rad/s, which with 5× separation puts the cable loop near 2 rad/s, below the 4.7 rad/s swing it is supposed to control.

ω_R	lag 35 ms: min damping	lag + 10 ms delay
10 rad/s	0.55	0.40
15 rad/s	0.36	0.20
20 rad/s	0.26	0.08
30 rad/s	0.13	unstable

Table 7: Least-damped pole of the attitude loop with motor lag.

New components and their gates (R1–R8). This step added the sensor and delay models, a body-rate controller and a single closed-loop runner. Noise statistics match the specification within 1%; the delay is exact; the new runner reproduces step 2.5 bit for bit when all effects are off; the rate loop's step response has a 17.5 ms time constant against 18.5 ms designed. The body-rate interface is mathematically equal to Paper 1's attitude law (to 10^{-14} N m) except for the feed-forward torque $JR^\top R_c\dot{\Omega}_c$, which a thrust-plus-rates command cannot carry: dropping it raises the T2 tracking error from 0.17 mm to about 3 mm (F9). The payload drag was set to a smooth sphere's $C_D = 0.47$, since MuJoCo's default formula gives an effective C_D of 1.0.

One factor at a time. Each effect was switched on alone, then all together, on the 11 feasible benchmark cases, with two gain sets: Paper 1's nominal 1.5 / 7.5 / 30 and a lag-compatible 0.4 / 2 / 10 rad/s. Noisy settings use 20 seeds, for 990 runs in total.

With the nominal gains, delay, drag and the body-rate interface cost little (91–100% success). Motor lag gives 82%: it breaks only the swing-recovery cases (errors of 45 and 170 m), while all eight tracking cases still succeed. Sensor noise is decisive: *all 220 noisy runs crash*, and so do all 220 runs with every effect on. A per-channel test at hover isolates the cause:

Paper 1 forms the cable rate as $\dot{q} = (v_L - v_Q)/l$ from two raw velocities, which gives about

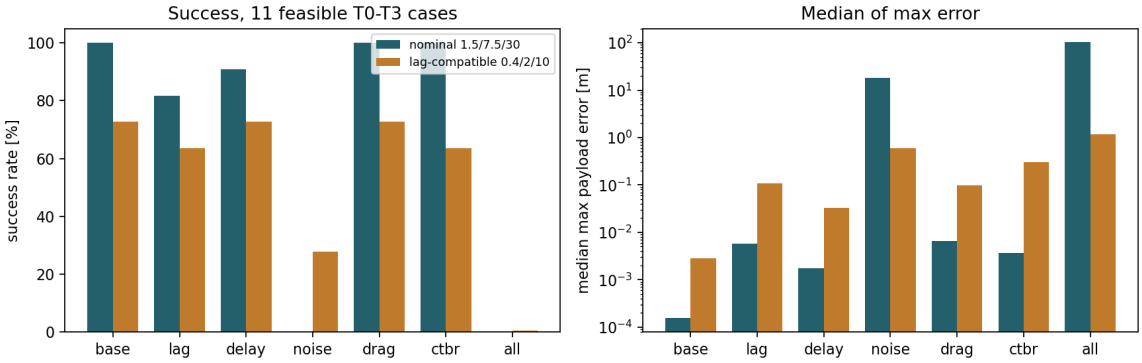


Figure 11: Success rate and median maximum error per setting: base (L0), motor lag 35 ms, delay 10 ms, sensor noise, drag, CTBR interface at 100 Hz, and all together.

Noise channel (D10 level)	Max error	Saturation	Outcome
Position 1 cm	0.41 m	40%	survives, poorly
Velocity 5 cm/s	2.4 m	96%	crash (tilt 173°)
Attitude 1°	2.4 mm	82%	survives
Gyro 0.05 rad/s	0.25 mm	9%	fine
All channels ×0.1	$\leq$ 0.7 mm	$\approx$ 0	fine

Table 8: Hover with one noise channel at a time, nominal gains.

0.14 rad/s of noise. The cable loop multiplies it and the 30 rad/s attitude loop turns it into large torque commands; the rotors saturate and the drone flips. The lag-compatible gains never crash, but they cannot damp a 30° swing even at L0 (3.5 m error), reach only 73% success at L0 and 0% with every effect on.

F10: realistic sensing and actuation

Under realistic sensing Paper 1 is unusable without a state estimator, and velocity noise is the main cause. Motor lag breaks swing recovery. The two cannot be fixed together by retuning: gains slow enough for the lag are too slow to damp the swing.

Caveat on the noise model

D10 applies white noise independently at every 500 Hz tick. Filtered motion-capture data is smoother at high frequency, so the *severity* of F10 depends on this assumption. The need for some filtering does not.

7 Summary of findings

ID	Finding
F1	MuJoCo air drag on the X2 is quadratic and small at low speed (0.10 N at 2 m/s); real rotor drag is roughly linear and larger. Not used as a precise model; covered by randomisation.
F2	Motor lag must use MuJoCo's `filter` dynamics under RK4 (`filterexact` has 1.6 cN error); read motor state from `data.act`.

ID	Finding
F3	Paper 1 needs about 5× loop separation; X2 rotors cap the attitude loop near 30 rad/s; gains 1.5 / 7.5 / 30 rad/s.
F4	X2 speed limit on Paper 1's ellipse: $T_0^* = 2.12$ s (0.5×1.0 m) and 2.61 s (1.0×2.0 m); at 1.5 s even unlimited thrust fails (bandwidth limit).
F5	Under rotor limits, recovery from flips $\geq 20°$ is chaotic and costs metres of height.
F6	For an inextensible slack cable, peak snap force is numerical; the snap impulse is physical and needs a 0.5 ms step.
F7	With a slack-capable cable Paper 1 tracks as before, survives a throw, but its 178° recovery diverges: it relied on the cable pushing.
F8	Fixed-bandwidth gains scale the cable force with length: 1.0 m crashes on a 15° swing; 0.25 m works.
F9	A body-rate (CTBR) interface drops Paper 1's feed-forward torque: 0.17 mm becomes about 3 mm.
F10	Velocity noise crashes Paper 1 (0/220); motor lag breaks swing recovery; lag-compatible gains cannot damp the swing.

Table 9: Findings of Phases 1 and 2. Full wording and numbers are in `SIMULATION_DESIGN.md`.

8 Discussion

8.1 What the results say about Paper 1

Paper 1's theory is correct and our implementation reproduces it to round-off where its assumptions hold. Its limitations are not errors but consequences of four assumptions that a real system violates: unlimited thrust, a cable that can push, perfect state knowledge and instantaneous actuators. The interesting result is how these assumptions interact through the time-scale separation the stability proof needs. Separation pushes the attitude loop up; rotor limits cap it at 30 rad/s; motor lag pushes it below about 10 rad/s; and the payload's own swing frequency sets a lower bound on the cable loop. On the X2 there is no set of cascade gains that satisfies all four.

8.2 Limits of this study

The study tests Paper 1 as published: gains were not retuned per condition, on purpose, and a systematic tuning protocol (D12) follows in Phase 3. The noise model is white per tick, as discussed above. The 35 ms motor time constant is a literature value from another platform; the X2's is not identified, and it will be randomised. Rotor downwash on the payload and ground effect are not modelled, as in the reference papers. All results are in simulation.

8.3 Gaps and next steps

The findings point to five capabilities that Paper 1 lacks: control that respects rotor limits (F3–F5), recovery that knows the cable can only pull (F7), state estimation (F10), gains or structures that hold across cable lengths (F8), and swing damping that is compatible with motor lag (F10). Phase 3 turns the survey (Paper 2) into baseline controllers and the tuning protocol; Phase 4 tests Paper 3, whose LQR with output weighting and Kalman filter on an elastic cable addresses part of these gaps directly. A contribution can be built on the combination that none of the

reviewed controllers covers: actuator- and slack-aware control with estimation, evaluated on this benchmark.

A Reproducing the results

All commands run from the repository root with the system Python 3 (MuJoCo 3.17, NumPy, Matplotlib, PyYAML). Each experiment creates `results/<phase>/<experiment>/<timestamp>/` with `config.yaml`, `meta.json` (including the git commit), `metrics.json`, `data.npz`, figures and a README.

Command	Produces
`scripts/test.sh`	all 66 tests (about 6 minutes)
`python3 experiments/e11_x2_validation.py`	drone gates G1–G5
`python3 experiments/e12_actuator_validation.py`	actuator gates A1–A6
`python3 experiments/e13_c1_p1_validation.py`	cable C1 gates P1–P7
`python3 experiments/e14_trajectories_metrics.py`	trajectory and metric gates M1–M5
`python3 experiments/e21_flatness.py`	flatness gates F1–F6
`python3 experiments/e22_geometric_control.py`	controller gates K1–K7
`python3 experiments/e23_ellipse_study.py`	E1.1, Figs. 4, 5
`python3 experiments/e24_recovery.py`	E1.2, Figs. 6, 7
`python3 experiments/e25_benchmark_subset.py`	E1.3 results table
`python3 experiments/e26a_c2_validation.py`	C2 gates S1–S6
`python3 experiments/e26a_paper1_on_c2.py`	E1.4a
`python3 experiments/e26b_cable_length.py`	E1.4b
`python3 experiments/e26c1_l1_validation.py`	L1 gates R1–R8
`python3 experiments/e26c2_realism_ofat.py -seeds=20 -workers=3`	E1.4c, 990 runs (about 2 hours on 3 cores; resumes from a checkpoint if stopped)

B Repository map

Path	Content
`SIMULATION_DESIGN.md`	design decisions D1–D12, review log, findings F1–F10, sources
`IMPLEMENTATION_PLAN.md`	working rules, step plan, progress log
`configs/base.yaml`	every parameter, each traced to a decision
`assets/skydio_x2/`	vendored Menagerie X2 files with licence
`sim/model.py`	builds the MuJoCo model (X2, cable C1/C2, payload, drag, motor lag)
`sim/actuators.py`, `sim/simulator.py`	mixer with PX4 priorities; multi-rate stepping
`sim/state.py`	state in Paper 1 notation, signed cable tension
`sim/trajectories.py`, `sim/jet.py`, `sim/flatness.py`	references with exact derivatives; flatness map
`sim/controllers/geometric_paper1.py`	Paper 1 controller
`sim/controllers/rate.py`, `sim/realism.py`, `sim/closed_loop.py`	CTBR rate loop; noise and delay; full-stack runner
`sim/checks_*.py`, `tests/`	validation gates and tests
`sim/controllers/blf_antiswing.py`	Ding, Sun & He (2023) barrier-Lyapunov controller, contributed by L. Alsheikh (Phase 3 work, not part of this report)

C Glossary

Underactuated fewer independent inputs than degrees of freedom; here the cable angles have no input.

Differential flatness all states and inputs are functions of a few outputs and their derivatives.

Geometric control control designed directly on the manifolds $SO(3)$ and S^2, avoiding Euler-angle singularities.

Time-scale separation nested loops with well separated bandwidths, so that inner loops can be treated as instantaneous.

CTBM / CTBR collective thrust with body moments / with body rates; two standard command interfaces.

Mixer maps thrust and moments to individual rotor thrusts and handles saturation.

Feasibility boundary T_0^* shortest ellipse period whose flatness-required rotor thrusts all lie in 0–13 N.

Dynamic swing angle between the actual cable direction and the one the flatness map predicts for the reference.

References

[1] K. Sreenath, T. Lee, V. Kumar, "Geometric control and differential flatness of a quadrotor UAV with a cable-suspended load," *IEEE Conference on Decision and Control*, 2013.

[2] T. Lee, M. Leok, N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," *IEEE Conference on Decision and Control*, 2010.

[3] E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," *IEEE/RSJ IROS*, 2012.

[4] MuJoCo Menagerie: a collection of high-quality simulation models for MuJoCo, Google DeepMind, `https://github.com/google-deepmind/mujoco_menagerie`.

[5] M. Faessler, A. Franchi, D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE Robotics and Automation Letters*, 2018.

[6] E. Kaufmann, L. Bauersfeld, D. Scaramuzza, "A benchmark comparison of learned control policies for agile quadrotor flight," *IEEE ICRA*, 2022.

[7] Tolba, Shirinzadeh, *Robotics and Autonomous Systems* 193 (2025) 105060 (Paper 3).

[8] F. Ding, C. Sun, S. He, "Anti-swing control for quadrotor-slung load transportation system with underactuated state constraints," *Sensors* 23 (2023) 8995.